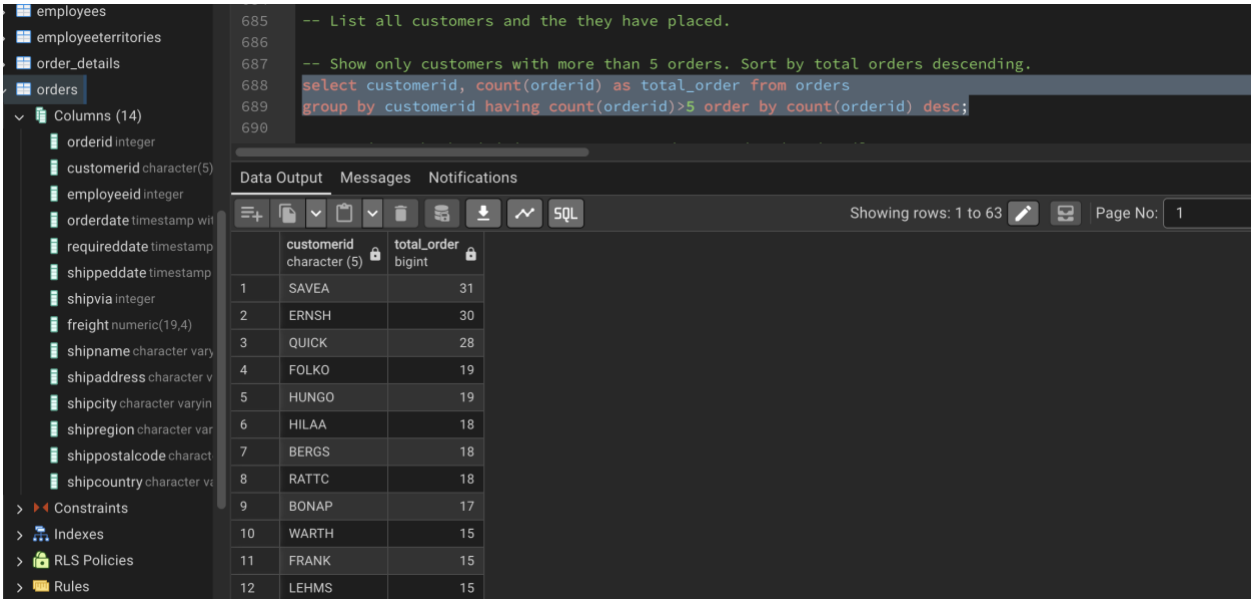
◈ BASIC TO INTERMEDIATE

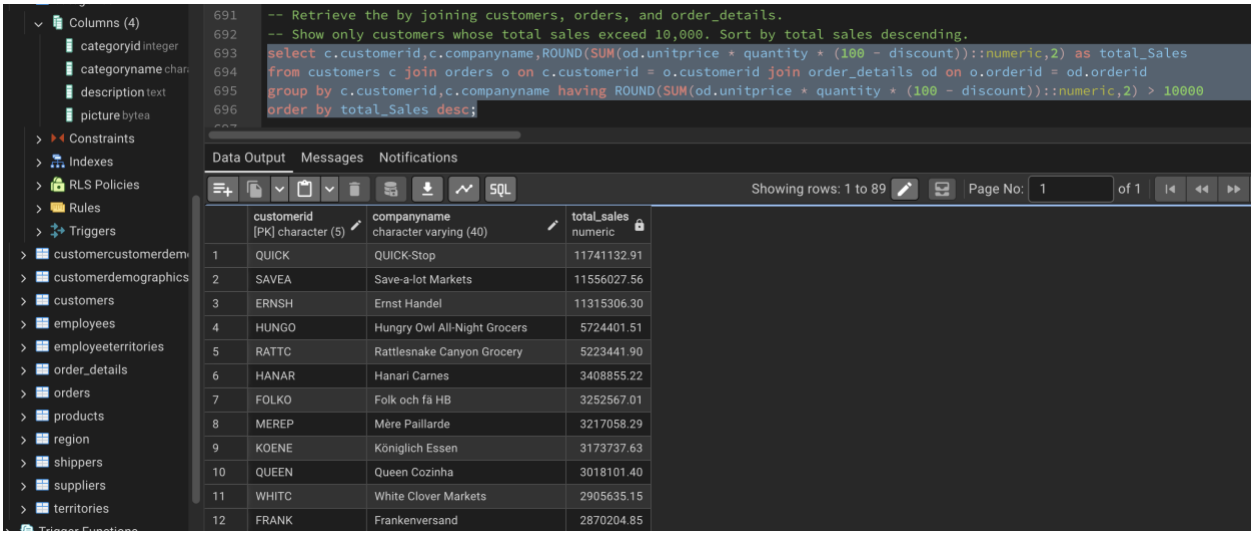
BASIC TO INTERMEDIATE
1. List all customers and the total number of orders they have placed. Show only customers with more than 5 orders. Sort by total orders descending.

```
select customerid, count(orderid) as total_order from orders
group by customerid having count(orderid)>5 order by count(orderid) desc;
```



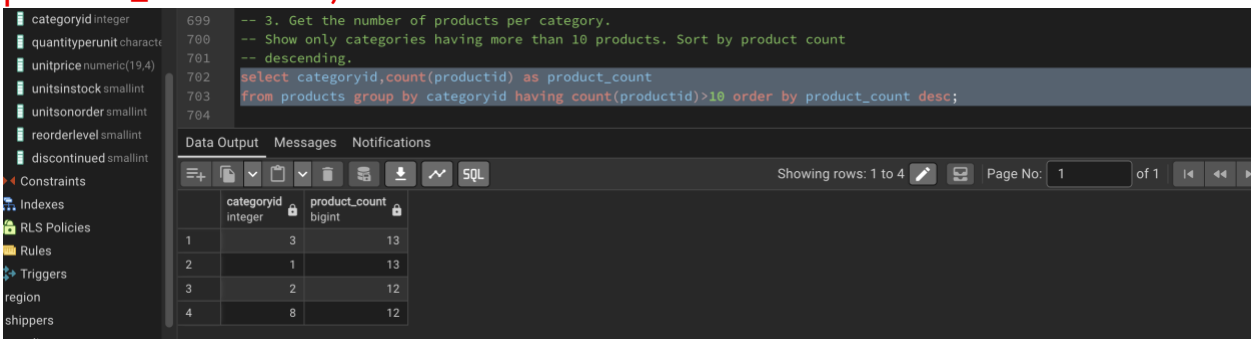
2. Retrieve the total sales amount per customer by joining customers, orders,
and order_details.
Show only customers whose total sales exceed 10,000. Sort by total sales descending.

```
select c.customerid,c.companyname,ROUND(SUM(od.unitprice * quantity *
(100 - discount))::numeric,2) as total_Sales from customers c join orders o
on c.customerid = o.customerid join order_details od on o.orderid =
od.orderid group by c.customerid,c.companyname having
ROUND(SUM(od.unitprice * quantity * (100 - discount))::numeric,2) > 10000
order by total_Sales desc;
```



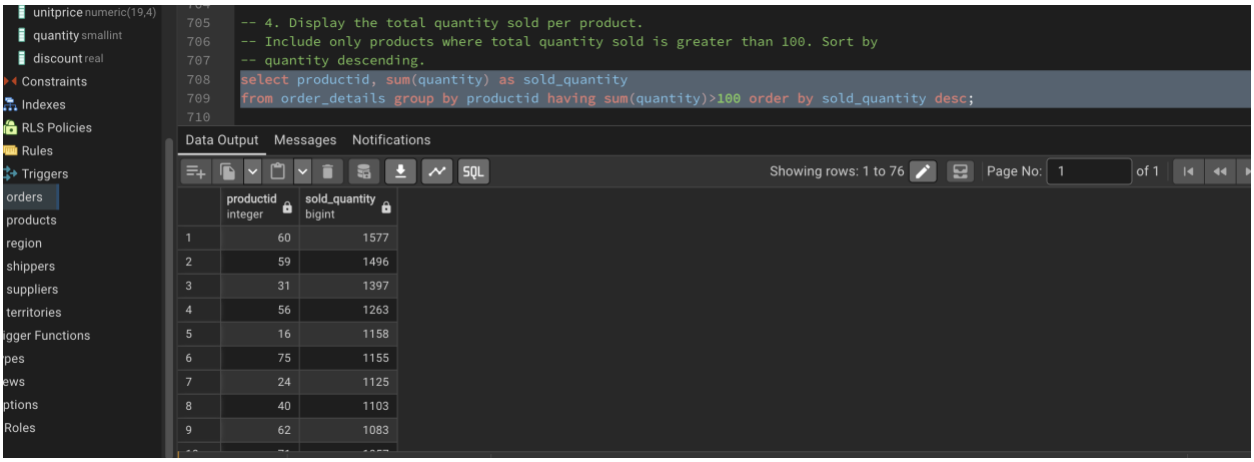
3. Get the number of products per category.
Show only categories having more than 10 products. Sort by product count descending.

select categoryid,count(productid) as product_count
from products group by categoryid having count(productid)>10 order by product_count desc;



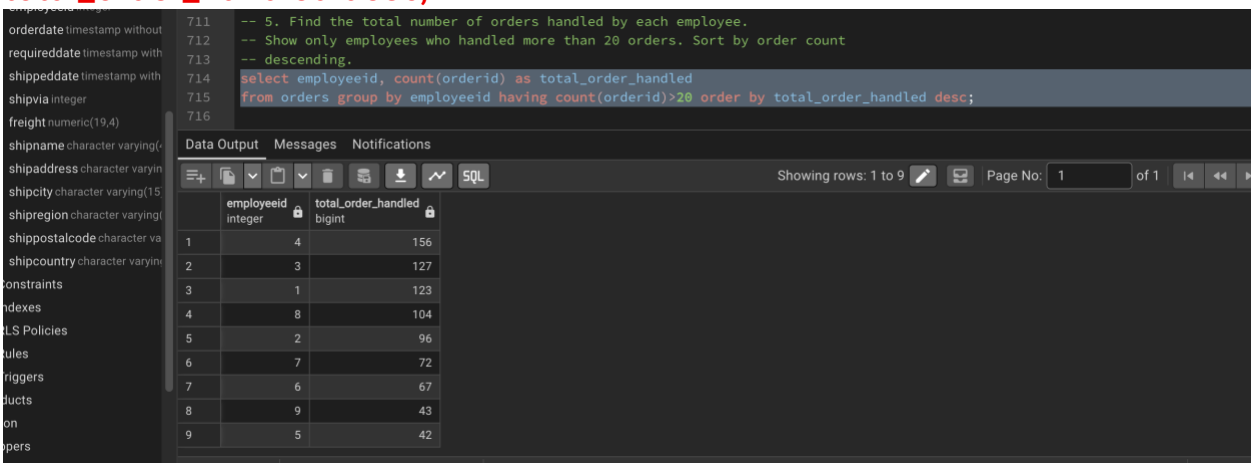
4. Display the total quantity sold per product.
Include only products where total quantity sold is greater than 100. Sort by quantity descending.

select productid, sum(quantity) as sold_quantity
from order_details group by productid having sum(quantity)>100 order by sold_quantity desc;



5. Find the total number of orders handled by each employee.
Show only employees who handled more than 20 orders. Sort by order count
descending.

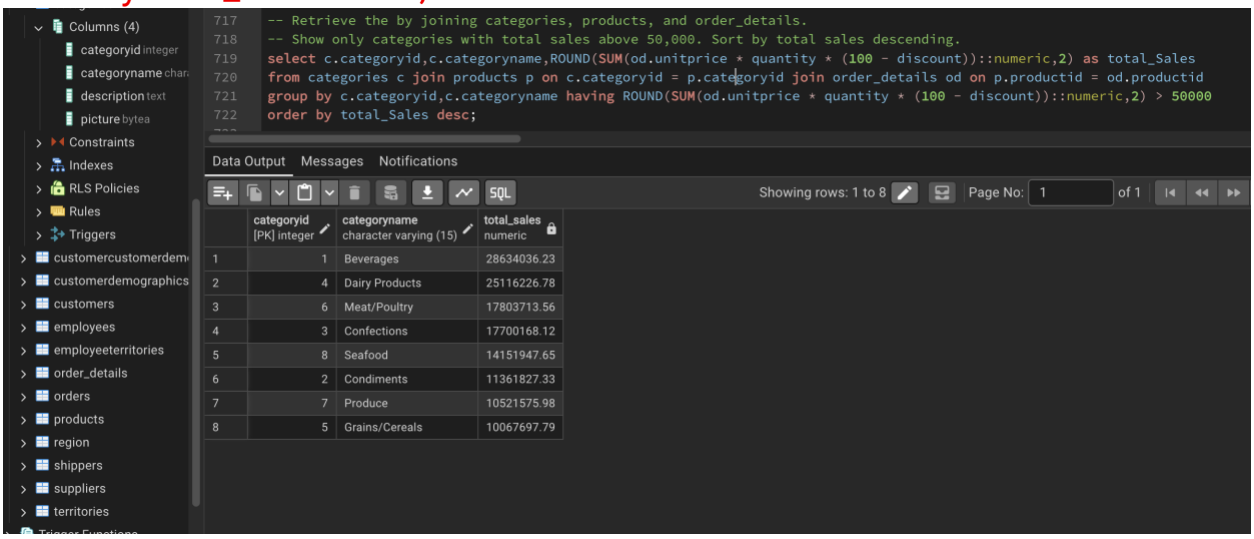
select employeeid, count(orderid) as total_order_handled
from orders group by employeeid having count(orderid)>20 order by total_order_handled desc;



◈ INTERMEDIATE

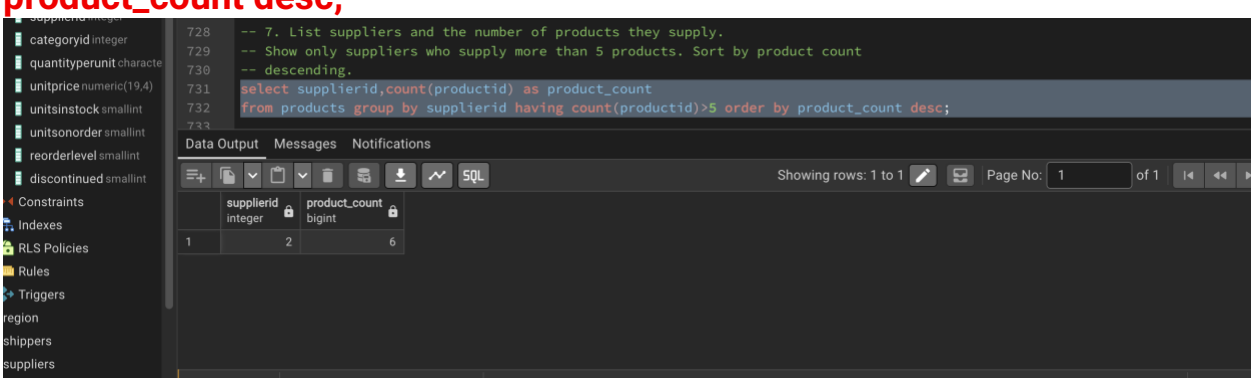
6. Retrieve the total sales per category by joining categories, products, and order_details.
Show only categories with total sales above 50,000. Sort by total sales descending.

select c.categoryid,c.categoryname,ROUND(SUM(od.unitprice * quantity * (100 - discount))::numeric,2) as total_Sales
from categories c join products p on c.categoryid = p.categoryid join order_details od on p.productid = od.productid
group by c.categoryid,c.categoryname having ROUND(SUM(od.unitprice * quantity * (100 - discount))::numeric,2) > 50000
order by total_Sales desc;



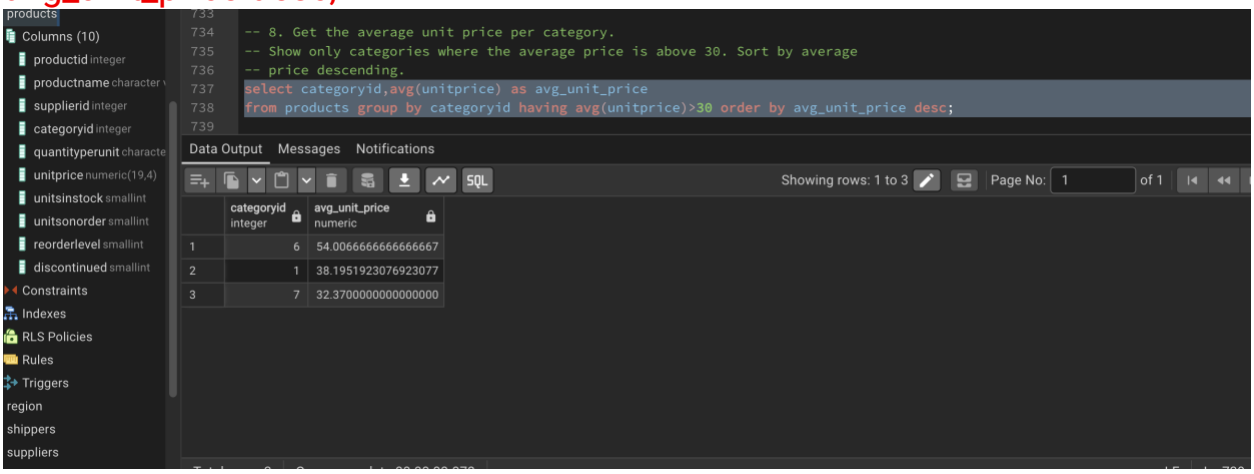
7. List suppliers and the number of products they supply.
Show only suppliers who supply more than 5 products. Sort by product count
descending.

select supplierid,count(productid) as product_count
from products group by supplierid having count(productid)>5 order by product_count desc;



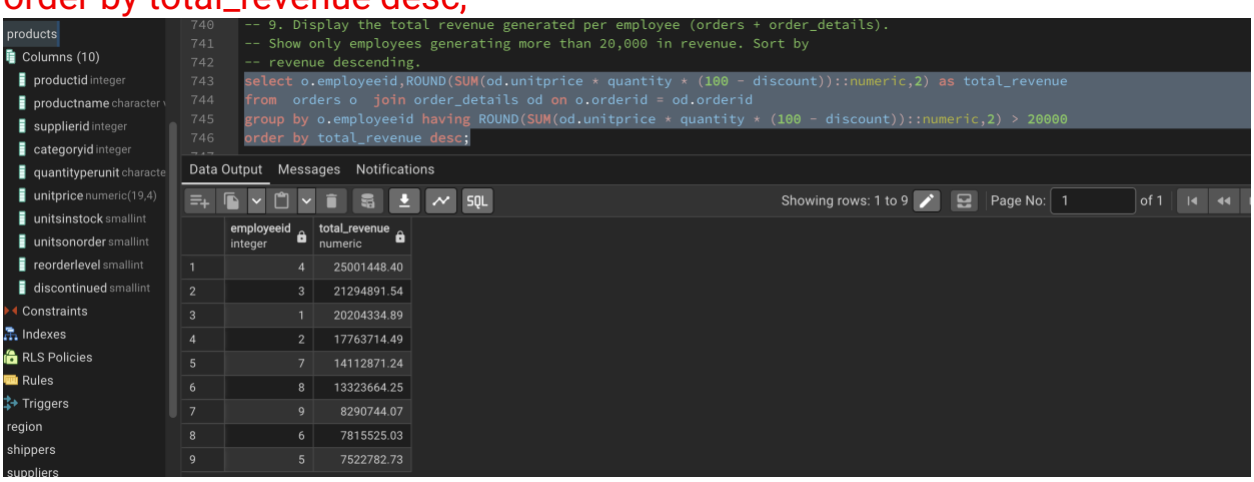
8. Get the average unit price per category.
Show only categories where the average price is above 30. Sort by average price descending.

select categoryid,avg(unitprice) as avg_unit_price
from products group by categoryid having avg(unitprice)>30 order by avg_unit_price desc;



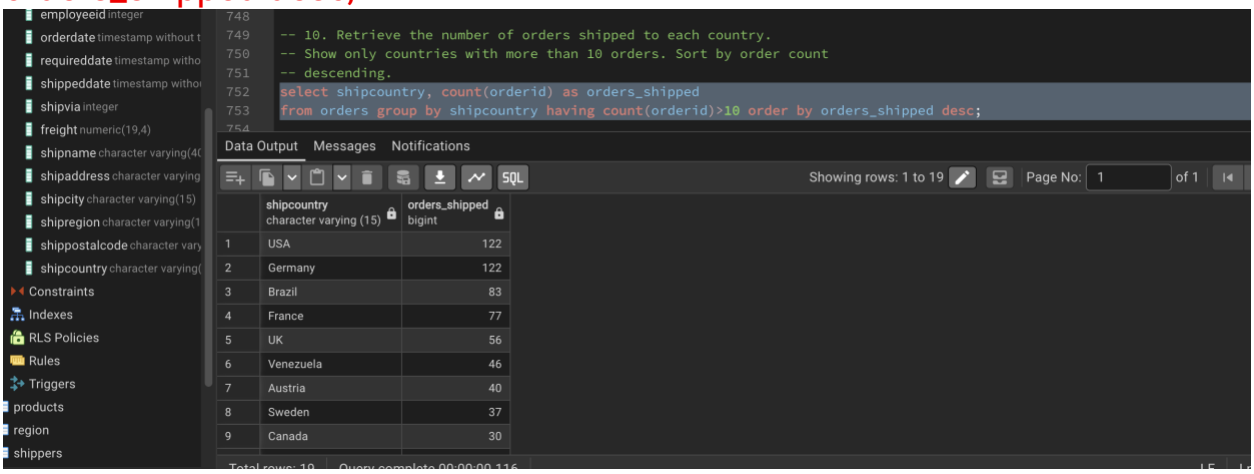
9. Display the total revenue generated per employee (orders + order_details).
Show only employees generating more than 20,000 in revenue. Sort by revenue descending.

select o.employeeid,ROUND(SUM(od.unitprice * quantity * (100 - discount))::numeric,2) as total_revenue
from orders o join order_details od on o.orderid = od.orderid
group by o.employeeid having ROUND(SUM(od.unitprice * quantity * (100 - discount))::numeric,2) > 20000
order by total_revenue desc;



10. Retrieve the number of orders shipped to each country.
Show only countries with more than 10 orders. Sort by order count descending.

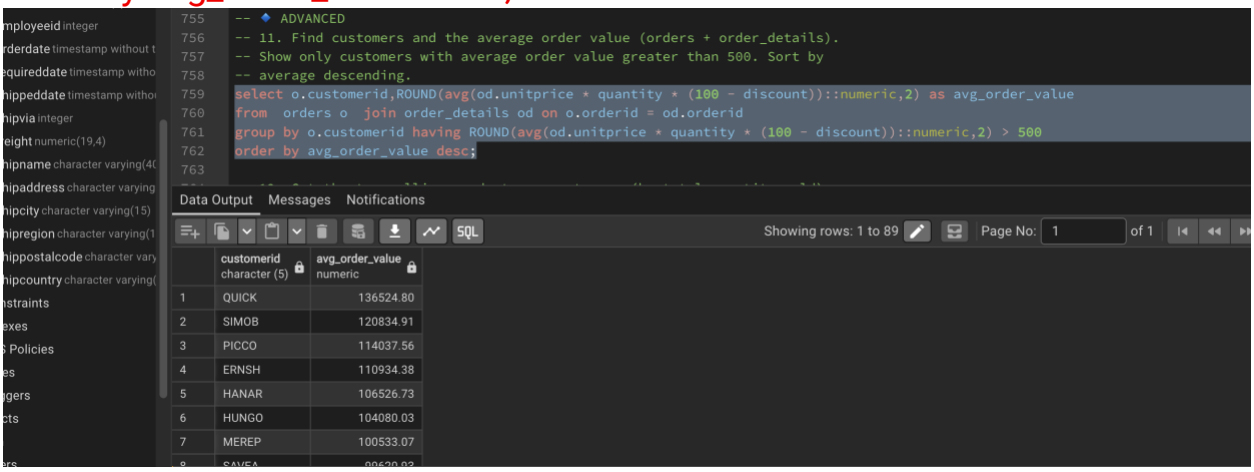
select shipcountry, count(orderid) as orders_shipped
from orders group by shipcountry having count(orderid)>10 order by orders_shipped desc;



◆◆ ADVANCED

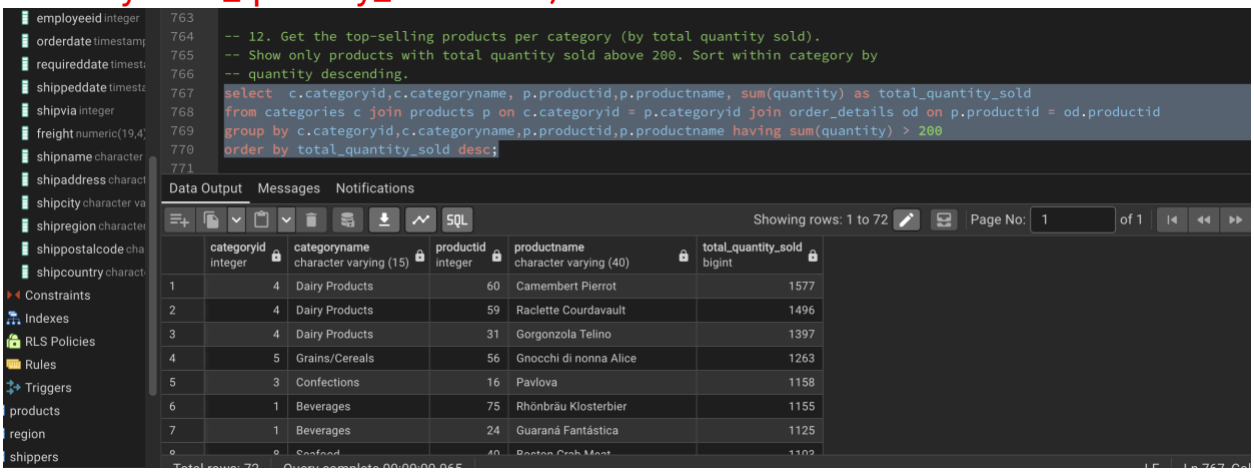
11. Find customers and the average order value (orders + order_details). Show only customers with average order value greater than 500. Sort by average descending.

select o.customerid,ROUND(avg(od.unitprice * quantity * (100 - discount))::numeric,2) as avg_order_value
from orders o join order_details od on o.orderid = od.orderid
group by o.customerid having ROUND(avg(od.unitprice * quantity * (100 - discount))::numeric,2) > 500
order by avg_order_value desc;



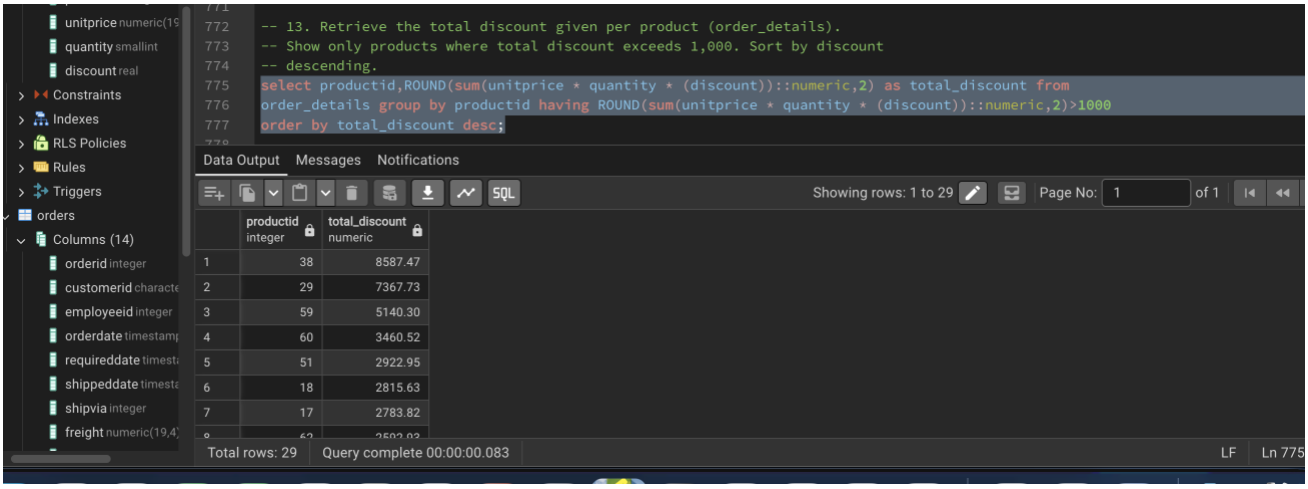
12. Get the top-selling products per category (by total quantity sold).
Show only products with total quantity sold above 200. Sort within category by
quantity descending.

```
select  c.categoryid,c.categoryname, p.productid,p.productname,
sum(quantity) as total_quantity_sold
from categories c join products p on c.categoryid = p.categoryid join
order_details od on p.productid = od.productid
group by c.categoryid,c.categoryname,p.productid,p.productname having
sum(quantity) > 200
order by total_quantity_sold desc;
```



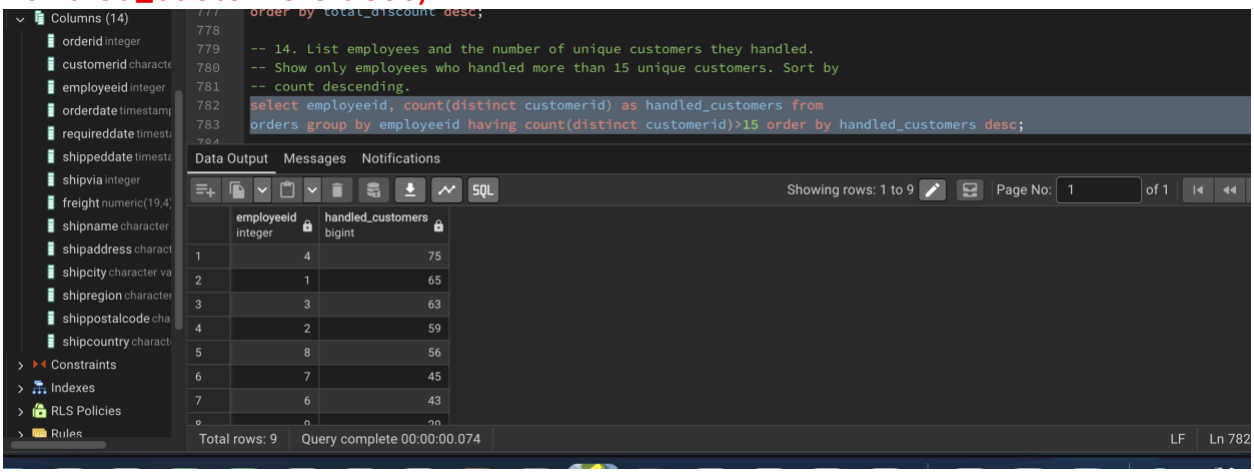
13. Retrieve the total discount given per product (order_details).
Show only products where total discount exceeds 1,000. Sort by discount descending.

```
select productid,ROUND(sum(unitprice * quantity * (discount))::numeric,2)
as total_discount from
order_details group by productid having ROUND(sum(unitprice * quantity *
(discount))::numeric,2)>1000
order by total_discount desc;
```



14. List employees and the number of unique customers they handled. Show only employees who handled more than 15 unique customers. Sort by count descending.

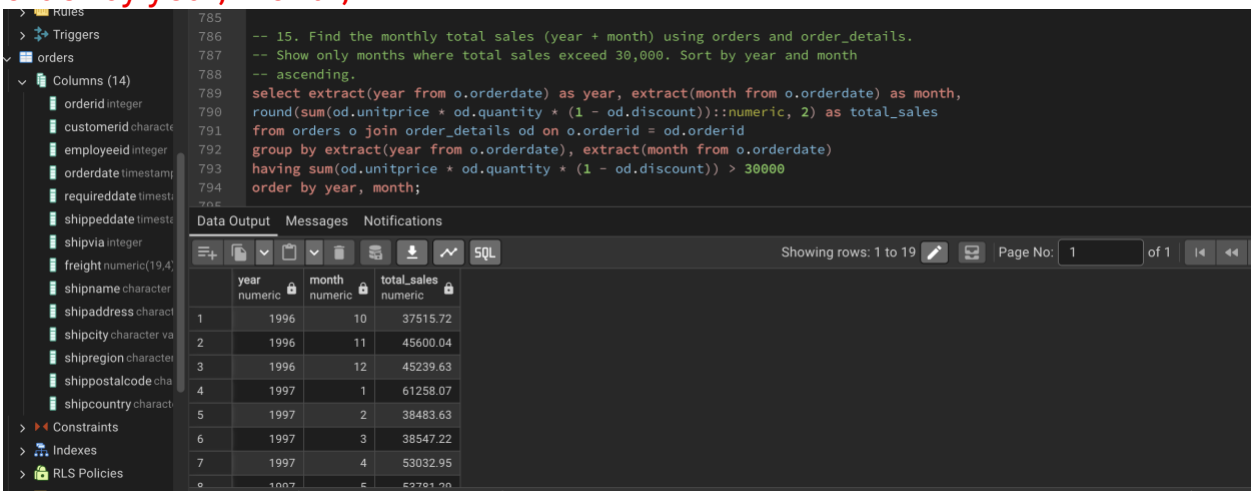
select employeeid, count(distinct customerid) as handled_customers from orders group by employeeid having count(distinct customerid)>15 order by handled_customers desc;



15. Find the monthly total sales (year + month) using orders and order_details.
Show only months where total sales exceed 30,000. Sort by year and month
ascending.

select extract(year from o.orderdate) as year, extract(month from o.orderdate) as month,
round(sum(od.unitprice * od.quantity * (1 - od.discount))::numeric, 2) as total_sales
from orders o join order_details od on o.orderid = od.orderid

group by extract(year from o.orderdate), extract(month from o.orderdate)
having sum(od.unitprice * od.quantity * (1 - od.discount)) > 30000
order by year, month;



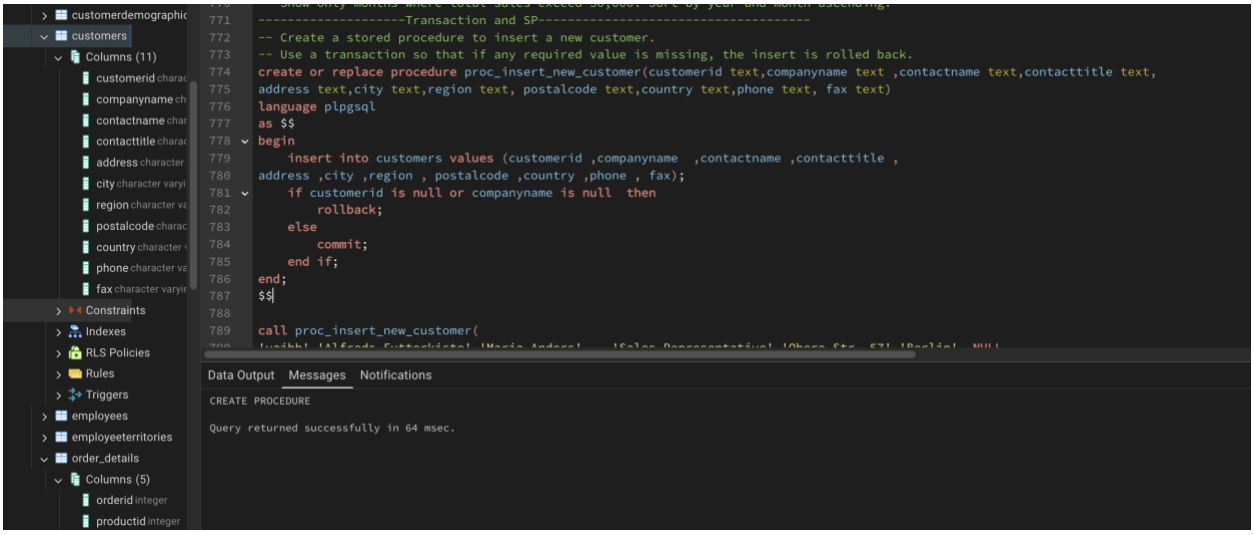
.---------- ----------Transaction and SP----------
----------Create a stored procedure to insert a new customer. Use a transaction so that if any required value is missing, the insert is rolled back.

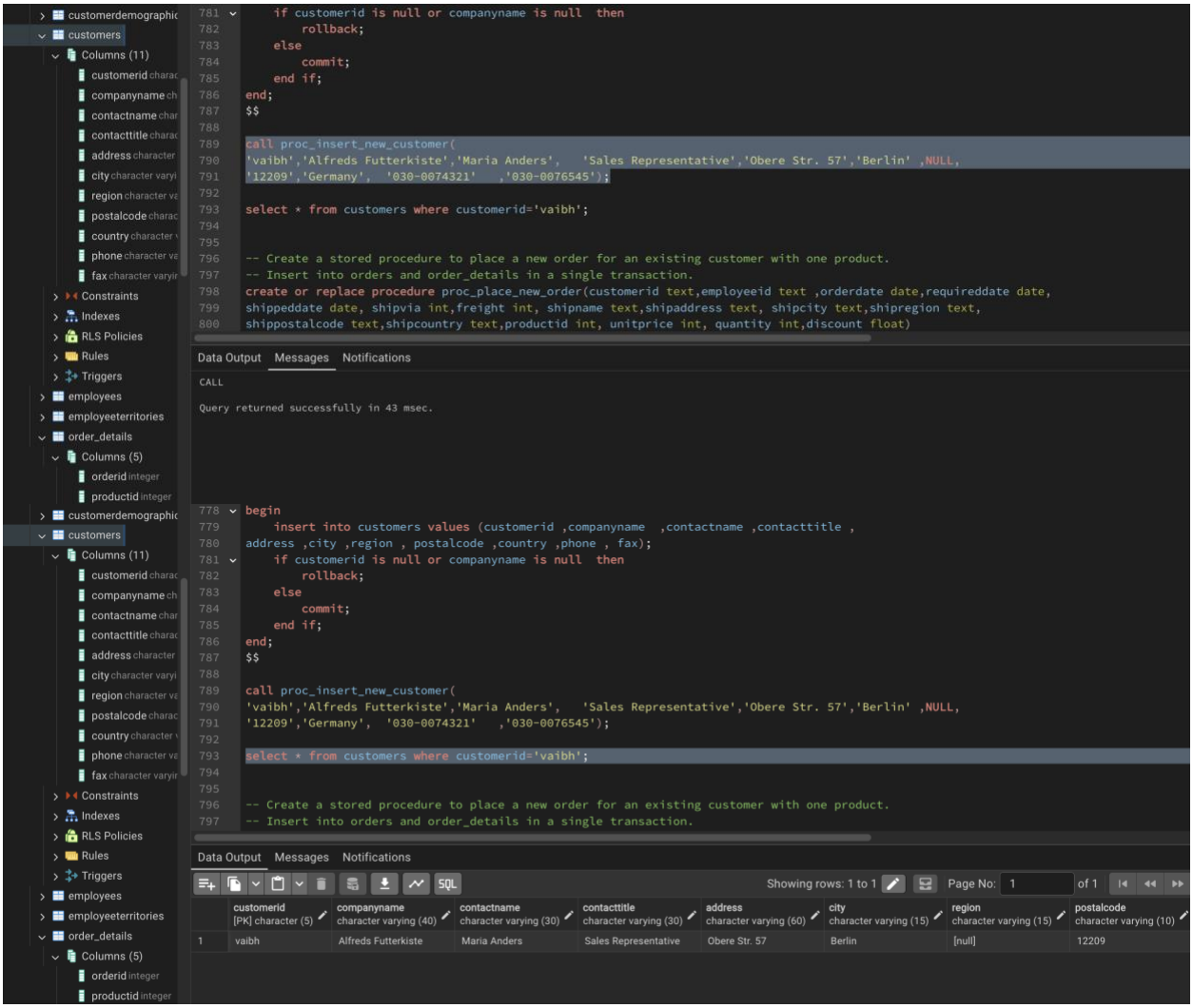
```
create or replace procedure proc_insert_new_customer(customerid
text,companyname text ,contactname text,contacttitle text,
address text,city text,region text, postalcode text,country text,phone text,
fax text)
language plpgsql
as $$
begin
	insert into customers values (customerid ,companyname
,contactname ,contacttitle ,
address ,city ,region , postalcode ,country ,phone , fax);
	if customerid is null or companyname is null  then
		rollback;
	else
		commit;
	end if;
end;
```

$$

call proc_insert_new_customer(
'vaibh','Alfreds Futterkiste','Maria Anders', 'Sales Representative','Obere Str. 57','Berlin' ,NULL,
'12209','Germany', '030-0074321' ,'030-0076545');

select * from customers where customerid='vaibh';

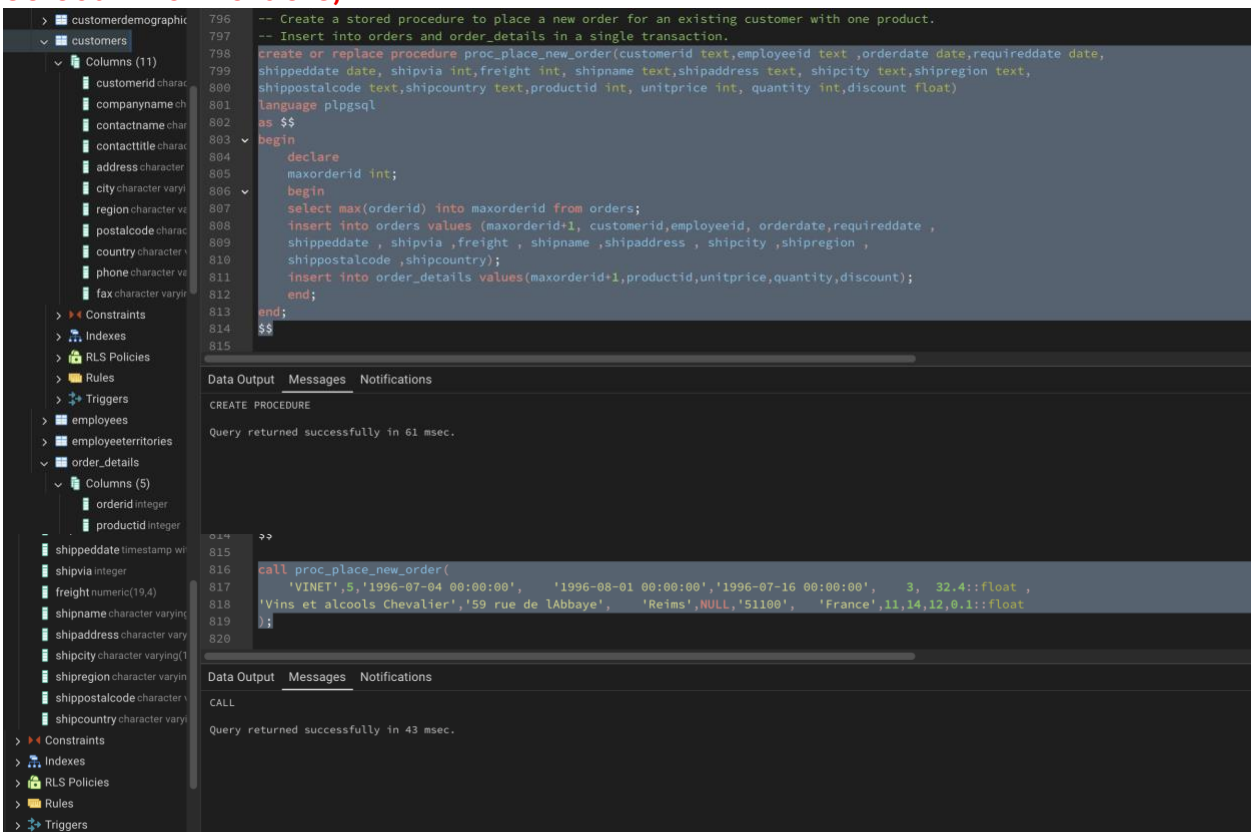




Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

```
create or replace procedure proc_place_new_order(customerid
text,employeeid text ,orderdate date,requireddate date,
shippeddate date, shipvia int,freight int, shipname text,shipaddress text,
shipcity text,shipregion text,
shippostalcode text,shipcountry text,productid int, unitprice int, quantity
int,discount float)
language plpgsql
as $$
begin
        declare
        maxorderid int;
```

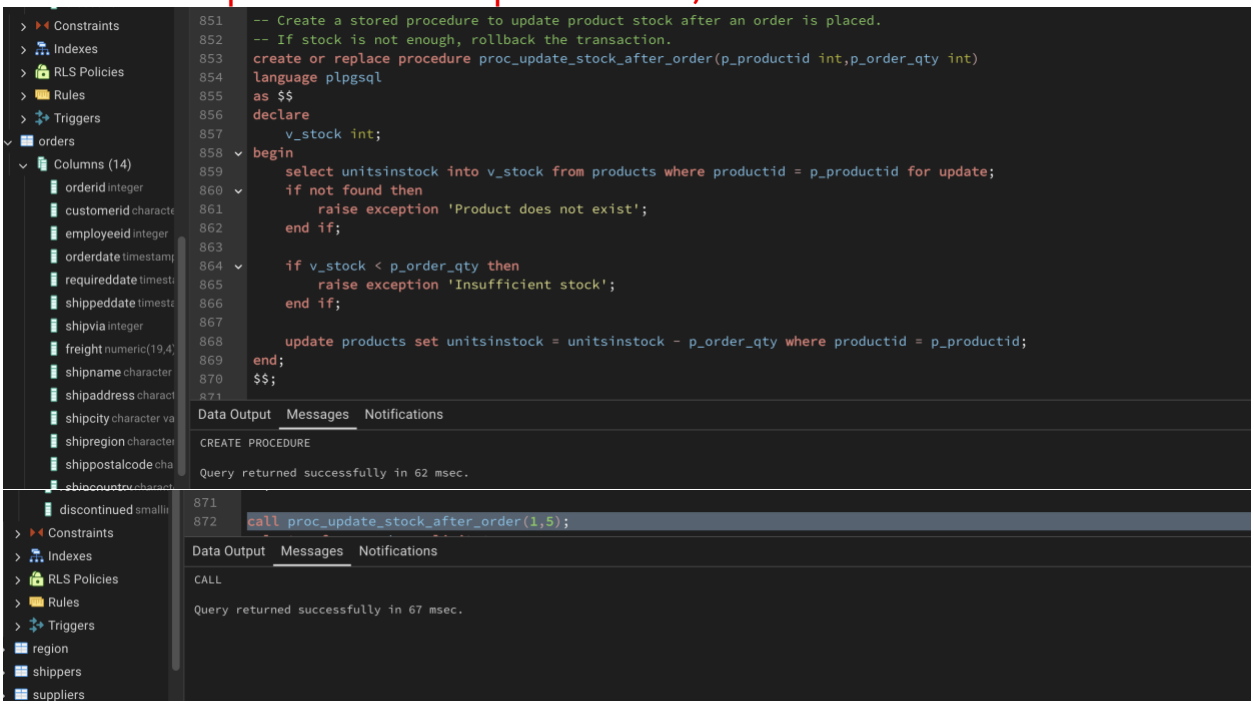
```
        begin
        select max(orderid) into maxorderid from orders;
        insert into orders values (maxorderid+1, customerid,employeeid,
orderdate,requireddate ,
        shippeddate , shipvia ,freight , shipname ,shipaddress , shipcity
,shipregion ,
        shippostalcode ,shipcountry);
        insert into order_details
values(maxorderid+1,productid,unitprice,quantity,discount);
        end;
end;
$$
-- call proc_place_new_order();
select * from orders;
```

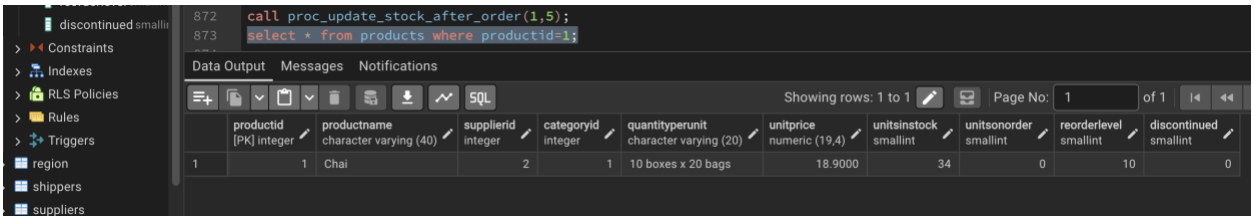


Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

```
create or replace procedure proc_update_stock_after_order(p_productid
int,p_order_qty int)
```

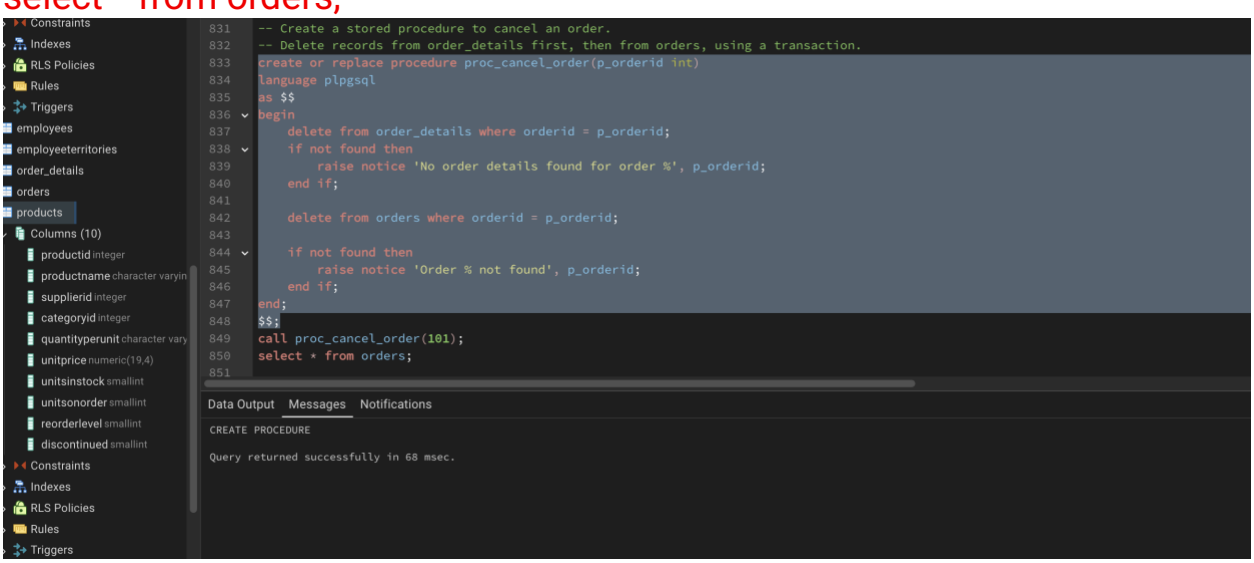
```
language plpgsql
as $$
declare
   v_stock int;
begin
   select unitsinstock into v_stock from products where productid =
p_productid for update;
   if not found then
      raise exception 'Product does not exist';
   end if;

   if v_stock < p_order_qty then
      raise exception 'Insufficient stock';
   end if;

   update products set unitsinstock = unitsinstock - p_order_qty where
productid = p_productid;
end;
$$;

call proc_update_stock_after_order(1,5);
select * from products where productid=1;
```

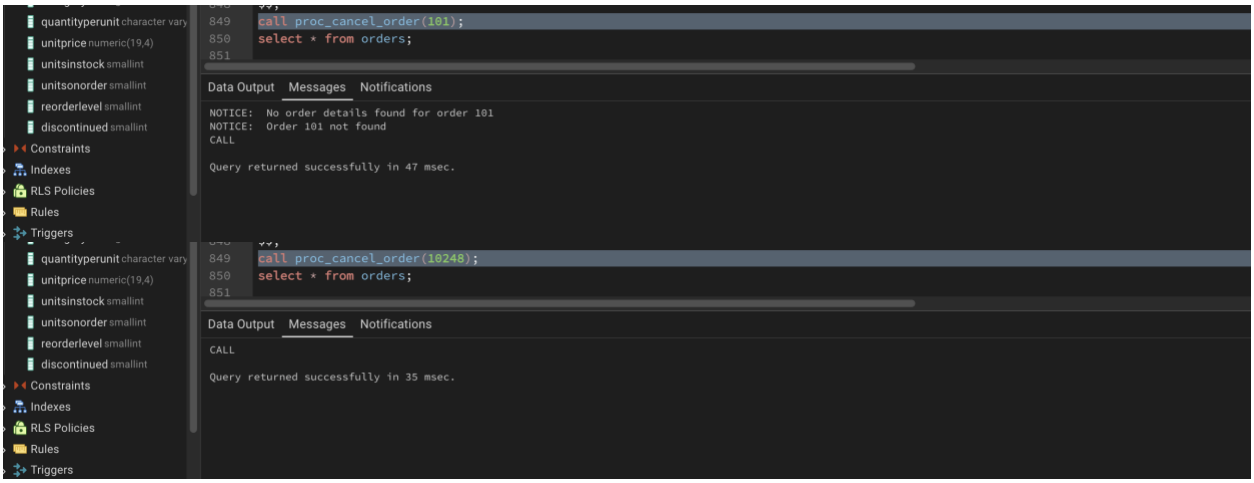




Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.

```
create or replace procedure proc_cancel_order(p_orderid int)
language plpgsql
as $$
begin
   delete from order_details where orderid = p_orderid;
   if not found then
      raise notice 'No order details found for order %', p_orderid;
   end if;

   delete from orders where orderid = p_orderid;

   if not found then
      raise notice 'Order % not found', p_orderid;
   end if;
end;
$$;
call proc_cancel_order(10248);
select * from orders;
```





Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

```
create or replace procedure proc_transfer_product(from_supplierid int, to_supplierid int, p_productid int)
language plpgsql
as $$
begin
   if not exists (
      select 1 from suppliers where supplierid = from_supplierid
   ) then
      raise exception 'Old supplier does not exist';
   end if;

   if not exists (
      select 1 from suppliers where supplierid = to_supplierid
   ) then
      raise exception 'New supplier does not exist';
   end if;

   update products set supplierid = to_supplierid
   where productid = p_productid;

end;
$$;
call proc_transfer_product(101,102,103);
call proc_transfer_product(1,2,1);
select * from products where productid=1;
```

```
-- Create a stored procedure to transfer products from one supplier to another.
-- If the old supplier or new supplier does not exist, rollback.
create or replace procedure proc_transfer_product(from_supplierid int, to_supplierid int, p_productid int)
language plpgsql
as $$
begin
    if not exists (
        select 1 from suppliers where supplierid = from_supplierid
    ) then
        raise exception 'Old supplier does not exist';
    end if;

    if not exists (
        select 1 from suppliers where supplierid = to_supplierid
    ) then
        raise exception 'New supplier does not exist';
    end if;

    update products set supplierid = to_supplierid
    where productid = p_productid;

end;
$$;
```

```
CREATE PROCEDURE

Query returned successfully in 51 msec.
```

Total rows: Query complete 00:00:00.051 LF Ln 854, Col

```
call proc_transfer_product(101,102,103);
```

```
ERROR:  Old supplier does not exist
CONTEXT:  PL/pgSQL function proc_transfer_product(integer,integer,integer) line 6 at RAISE
```

Total rows: Query complete 00:00:00.049 LF Ln 876, Col

```
call proc_transfer_product(1,2,1);
select * from products where productid=1;
```

```
CALL

Query returned successfully in 62 msec.
```

```
select * from products where productid=1;
```

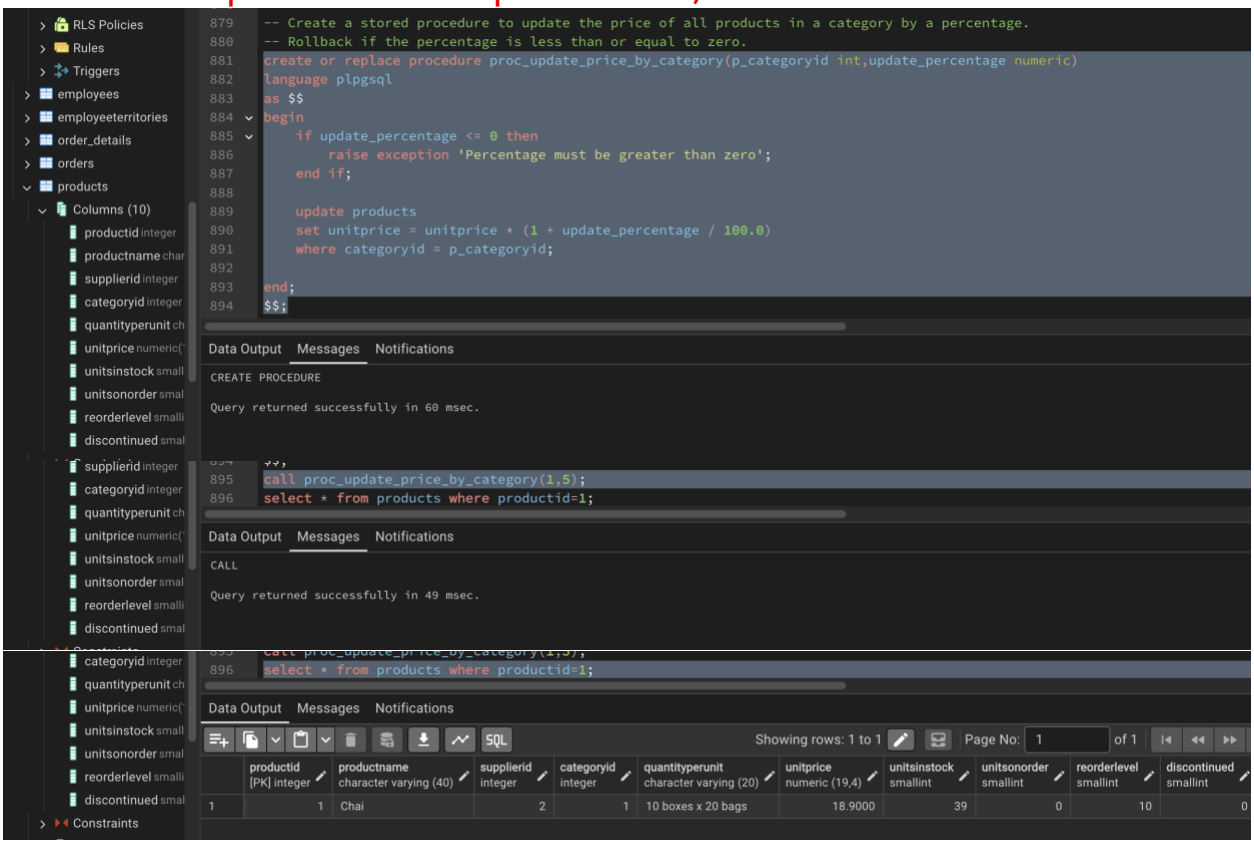
Showing rows: 1 to 1 Page No: 1 of 1

	productid [PK] integer	productname character varying (40)	supplierid integer	categoryid integer	quantityperunit character varying (20)	unitprice numeric (19,4)	unitsinstock smallint	unitsonorder smallint	reorderlevel smallint	discontinued smallint
1	1	Chai	2	1	10 boxes x 20 bags	18.0000	39	0	10	0

Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

```
create or replace procedure proc_update_price_by_category(p_categoryid int,update_percentage numeric)
language plpgsql
as $$
begin
    if update_percentage <= 0 then
        raise exception 'Percentage must be greater than zero';
    end if;

    update products
```

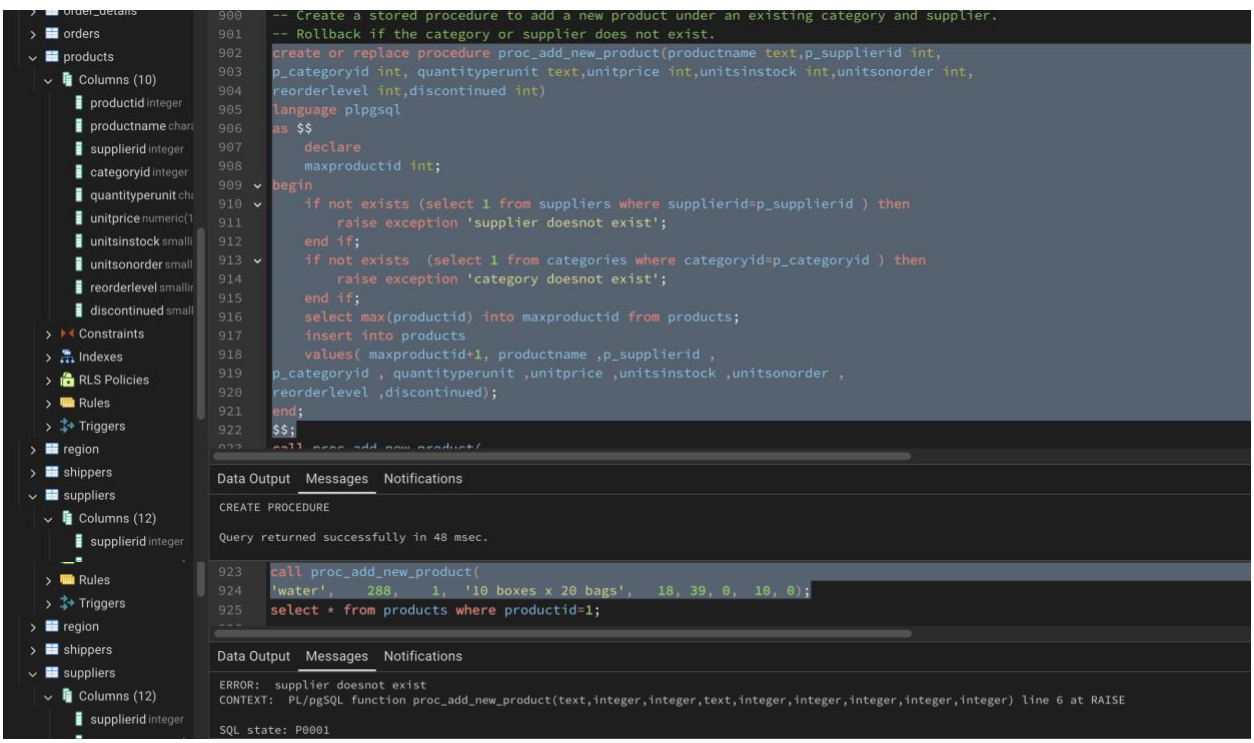
```
    set unitprice = unitprice * (1 + update_percentage / 100.0)
    where categoryid = p_categoryid;

end;
$$;
call proc_update_price_by_category(1,5);
select * from products where productid=1;
```

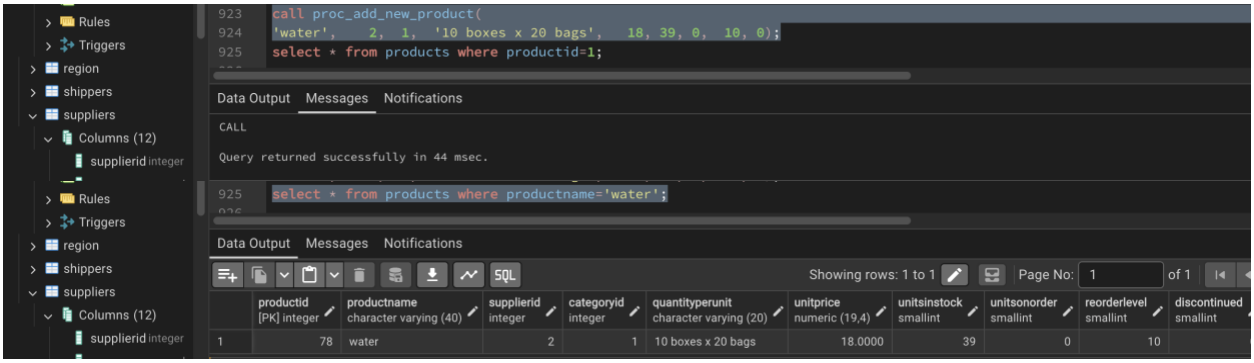


Create a stored procedure to add a new product under an existing category and supplier. Rollback if the category or supplier does not exist.

```
create or replace procedure proc_add_new_product(productname
text,p_supplierid int,
p_categoryid int, quantityperunit text,unitprice int,unitsinstock
int,unitsonorder int,
reorderlevel int,discontinued int)
language plpgsql
as $$
        declare
        maxproductid int;
begin
```

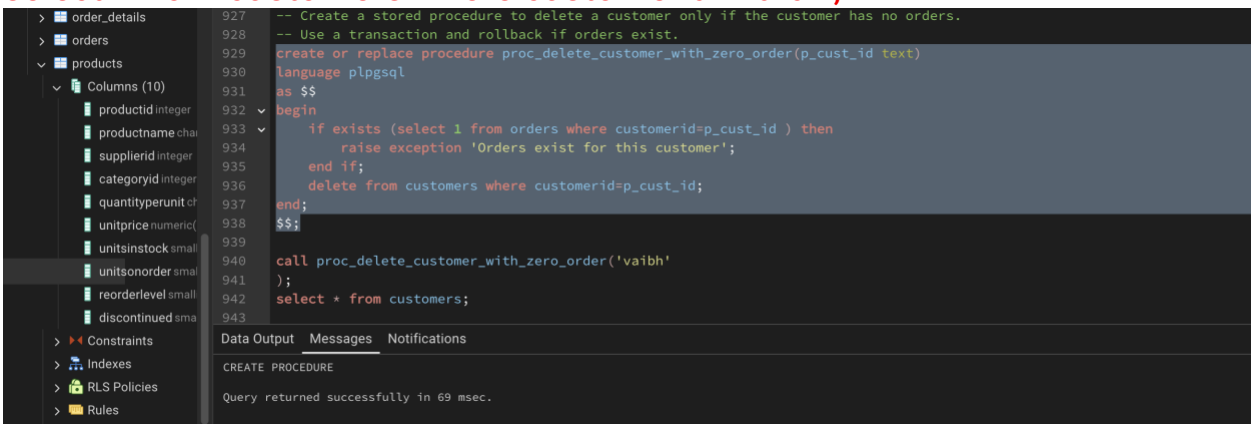
```
   if not exists (select 1 from suppliers where supplierid=p_supplierid ) then
      raise exception 'supplier doesnot exist';
   end if;
        if not exists  (select 1 from categories where
categoryid=p_categoryid ) then
      raise exception 'category doesnot exist';
   end if;
        select max(productid) into maxproductid from products;
   insert into products
   values( maxproductid+1, productname ,p_supplierid ,
p_categoryid , quantityperunit ,unitprice ,unitsinstock ,unitsonorder ,
reorderlevel ,discontinued);
end;
$$;
call proc_add_new_product(
'water',       2,       1,       '10 boxes x 20 bags',    18,    39,    0,       10,    0);
select * from products where productname='water';
```

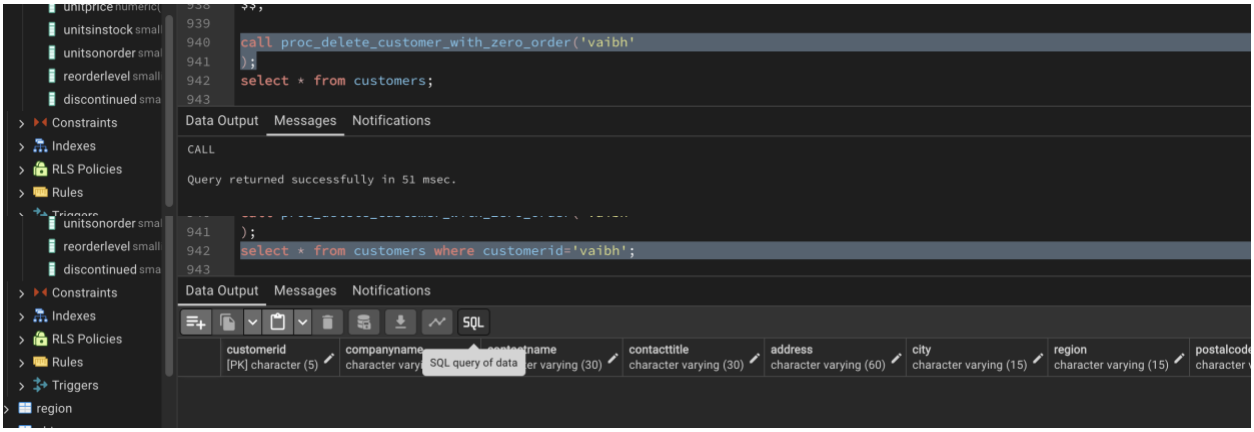




Create a stored procedure to delete a customer only if the customer has no orders. Use a transaction and rollback if orders exist.

```
create or replace procedure
proc_delete_customer_with_zero_order(p_cust_id text)
language plpgsql
as $$
begin
   if exists (select 1 from orders where customerid=p_cust_id ) then
      raise exception 'Orders exist for this customer';
   end if;
        delete from customers where customerid=p_cust_id;
end;
$$;

call proc_delete_customer_with_zero_order('vaibh'
);
select * from customers where customerid='vaibh';
```

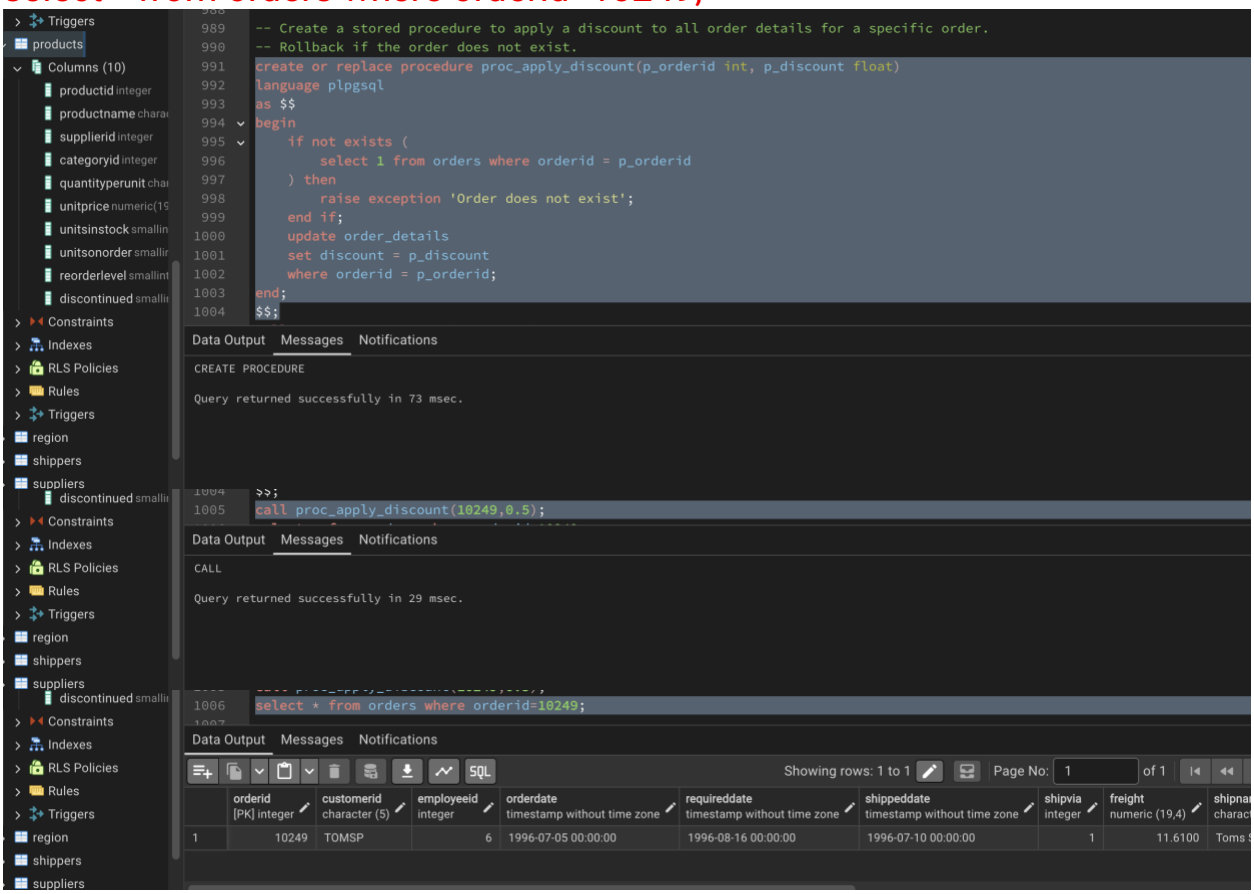




Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

```
create or replace procedure proc_apply_discount(p_orderid int, p_discount float)
language plpgsql
as $$
begin
   if not exists (
      select 1 from orders where orderid = p_orderid
   ) then
      raise exception 'Order does not exist';
   end if;
        update order_details
   set discount = p_discount
   where orderid = p_orderid;
end;
$$;
call proc_apply_discount(10249,0.5);
```

select * from orders where orderid=10249;



Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

```
create or replace procedure proc_place_order(p_customerid
text,p_productids int[],p_quantities int[])
language plpgsql
as $$
declare
   v_orderid int;
   i int;
   v_stock int;
begin
   insert into orders(customerid, orderdate)
   values (p_customerid, current_date)
   returning orderid into v_orderid;

        for i in 1 .. array_length(p_productids, 1)
```

```
    loop
        select unitsinstock
        into v_stock
        from products
        where productid = p_productids[i]
        for update;

        if not found then
            raise exception 'Product % does not exist', p_productids[i];
        end if;

        if v_stock < p_quantities[i] then
            raise exception 'Insufficient stock for product %', p_productids[i];
        end if;

        insert into order_details(orderid, productid, quantity, unitprice,
discount)
        select v_orderid, productid, p_quantities[i], unitprice, 0
        from products
        where productid = p_productids[i];
        update products
        set unitsinstock = unitsinstock - p_quantities[i]
        where productid = p_productids[i];
    end loop;
end;
$$;
CALL proc_place_order(
    'AFKI',                 -- customer id
    ARRAY[1, 2, 3],         -- product ids
    ARRAY[5, 10, 2]          -- quantities
);
SELECT *
FROM orders
ORDER BY orderid DESC
LIMIT 1;
```

```
$$;
CALL proc_place_order(
    'AFKI',                  -- customer id
    ARRAY[1, 2, 3],           -- product ids
    ARRAY[5, 10, 2]           -- quantities
);
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "pk_orders"
Key (orderid)=(11078) already exists.

SQL state: 23505
Detail: Key (orderid)=(11078) already exists.
Context: SQL statement "insert into orders(customerid, orderdate)
    values (p_customerid, current_date)
    returning orderid"
PL/pgSQL function proc_place_order(text,integer[],integer[]) line 7 at SQL statement
```

```
create or replace procedure proc_place_order(p_customerid text,p_productids int[],p_quantities int[])
language plpgsql
as $$
declare
    v_orderid int;
    i int;
    v_stock int;
begin
    insert into orders(customerid, orderdate)
    values (p_customerid, current_date)
    returning orderid into v_orderid;

    for i in 1 .. array_length(p_productids, 1)
    loop
        select unitsinstock
        into v_stock
        from products
        where productid = p_productids[i]
        for update;

        if not found then
            raise exception 'Product % does not exist', p_productids[i];
        end if;

        if v_stock < p_quantities[i] then
            raise exception 'Insufficient stock for product %', p_productids[i];
        end if;

        insert into order_details(orderid, productid, quantity, unitprice, discount)
        select v_orderid, productid, p_quantities[i], unitprice, 0
        from products
        where productid = p_productids[i];
        update products
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 63 msec.
```